

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 10 EXERCISES

STRUCTURED QUERY LANGUAGE (SQL):

DATA MANIPULATION LANGUAGE (DML)

BOOK STORE RELATION SCHEMAS

Author(authorId, firstName, lastName)

Book(bookId, title, subject, quantityInStock, price, *authorId*)

BookOrder(orderId, *customerId*, orderYear)

Customer(customerId, firstName, lastName)

OrderDetails(orderId, bookId, format, language, quantity)

Attribute names in italics are foreign key attributes.

Assumptions

Each author has authored at least one book in the store.

Each book has exactly one Author.

A book may not have sold any copies (i.e., there is no order for the book).

Each order is made by exactly one customer and has one or more associated tuples in OrderDetails (e.g., one order may contain several different books).

EXERCISE 1

Find each customer's rank with respect to their total number of orders.
Include in the query result tuples attributes first name, last name and customer rank.
Order the query result tuples first by rank ascending and then by last name ascending.

```
select firstName, lastName,  
       dense_rank() over (order by count(orderId) desc) as customerRank  
from BookOrder natural join Customer  
group by customerId, firstName, lastName  
order by customerRank, lastName;
```

customerId	firstName	lastName	count(orderId)
5	Carl	Ip	3
2	Sam	Lam	3
1	Harry	Lee	10
3	Ron	Lee	5
6	Lilly	Lee	1
4	Andrew	Chu	3



Query result

firstName	lastName	customerRank
Harry	Lee	1
Ron	Lee	2
Abdrew	Chu	3
Carl	Ip	3
Sam Lam	Lee	3
Lilly	Lee	4

Author(authorId, firstName, lastName) Book(bookId, title, subject, quantityInStock, price, *authorId*) BookOrder(orderId, *customerId*, orderYear)
Customer(customerId, firstName, lastName) OrderDetails(orderId, bookId, format, language, quantity)



EXERCISE 2

Find the orders of each customer.
Include in the query result tuples attributes last name, total quantity of the customer's orders and a comma-separated list in ascending order of the customer's order ids.
Order the query result tuples first by total quantity descending and then by last name ascending.

Is this a
correct
solution?
No! Why?

```
select lastName, sum(quantity) as totalQuantity  
      listagg(distinct orderId, ', ') within group (order by orderId) as orders  
from Customer natural join BookOrder natural join OrderDetails  
group by lastName  
order by totalQuantity desc, lastName;
```

What if there are two customers with the same last name?

Author(authorId, firstName, lastName)

Book(bookId, title, subject, quantityInStock, price, *authorId*)

BookOrder(orderId, *customerId*, orderYear)

Customer(customerId, firstName, lastName)

OrderDetails(orderId, bookId, format, language, quantity)

EXERCISE 2 (CONTD)

Find the orders of each customer.
Include in the query result tuples attributes last name, total quantity of the customer's orders and a comma-separated list in ascending order of the customer's order ids.
Order the query result tuples first by total quantity descending and then by last name ascending.

```
select lastName, sum(quantity) as totalQuantity,  
       listagg(distinct orderId, ', ' ) within group (order by orderId) as orders  
from Customer natural join BookOrder natural join OrderDetails  
group by lastName, customerId  
order by totalQuantity desc, lastName;
```

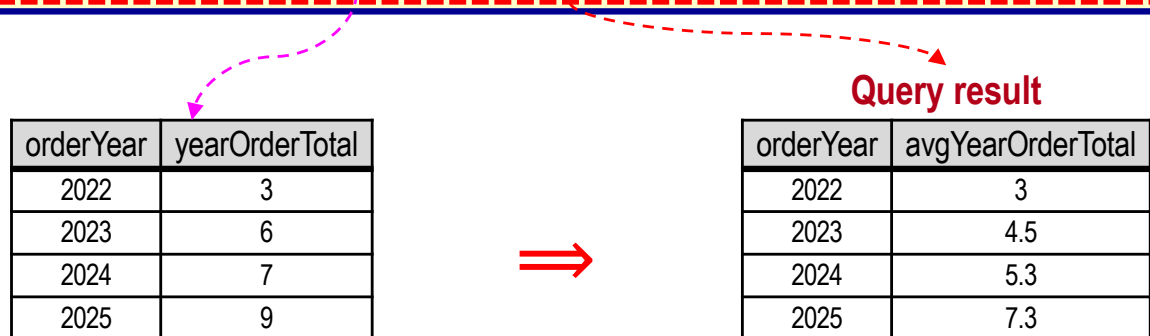
Query result

lastName	totalQuantity	orders
Chu	711	8, 9, 22
Lam	711	6, 15, 19
Lee	711	1, 2, 3, 4, 5, 12, 17, 18, 20, 23
Ip	584	11, 13, 24
Lee	584	7, 10, 14, 16, 21
Lee	53	25

EXERCISE 3

Find the average total number of orders for each year as well as the previous two years rounded to one decimal place.

```
select orderYear,  
       round(avg(yearOrderTotal) over (order by orderYear rows 2 preceding),1)  
       as avgYearOrderTotal  
from (select orderYear, count(*) as yearOrderTotal  
      from BookOrder  
      group by orderYear);
```



orderYear	yearOrderTotal
2022	3
2023	6
2024	7
2025	9

Query result

orderYear	avgYearOrderTotal
2022	3
2023	4.5
2024	5.3
2025	7.3

Author(authorId, firstName, lastName)

Book(bookId, title, subject, quantityInStock, price, *authorId*)

BookOrder(orderId, *customerId*, orderYear)

Customer(customerId, firstName, lastName)

OrderDetails(orderId, bookId, format, language, quantity)

EXERCISE 4

Find the authors who sold the three highest total number of books.
Include in the query result tuples attributes author name (first followed by last) and a comma-separated list in ascending order of the subjects on which they wrote their books.
Order the query result tuples by total books sold descending.

Is this a
correct
solution?

No!
Why?

```
select firstName || ' ' || lastName as author, subjects
from (select firstName, lastName, listagg(distinct subject, ', ')
      within group (order by subject) as subjects,
      dense_rank() over (order by sum(quantity) desc nulls last) as salesRank
from Author natural join Book natural left outer join OrderDetails
group by authorId, firstName, lastName
order by salesRank
fetch first 3 rows with ties)
order by salesRank;
```

Suppose the ranking
looks like this.
Do you see the problem
with this query?

author	salesRank
Pied Piper	1
Dan Brown	2
Michael Crichton	2
Donald Green	3
Seth Godin	4

The query will not get all the
authors with the three
highest number of books
sold if there are ties in the
top two authors. Why?

EXERCISE 4 (CONTD)

Find the authors who sold the three highest total number of books.
Include in the query result tuples attributes author name (first followed by last) and a comma-separated list in ascending order of the subjects on which they wrote their books.
Order the query result tuples by total books sold descending.

```
select firstName || ' ' || lastName as author, subjects
from (select firstName, lastName, listagg(distinct subject, ', ')
      within group (order by subject) as subjects,
      dense_rank() over (order by sum(quantity) desc nulls last) as salesRank
 from Author natural join Book natural left outer join OrderDetails
 group by authorId, firstName, lastName)
where salesRank<=3
order by salesRank;
```

Query Result

author	subjects
Pied Piper	Art, Business
Dan Brown	Art, Business, Fiction
Michael Crichton	Fiction
Donald Green	Business

EXERCISE 5

For all authors who wrote books on at least three subjects, increase the price of all their books by 5%.

```
update Book
set price=1.05*price
where authorId in (select authorId
from Book
group by authorId
having count(distinct subject)>=3);
```

Update the
price by 5%.

Authors who wrote books
on five or more subjects.

Author(authorId, firstName, lastName)

Book(bookId, title, subject, quantityInStock, price, *authorId*)

BookOrder(orderId, *customerId*, orderYear)

Customer(customerId, firstName, lastName)

OrderDetails(orderId, bookId, format, language, quantity)

EXERCISE 6

Find, for each subject, orders that include books on the subject.
Include in the query result tuples attributes subject, total quantity of books sold on the subject and a comma-separated list in ascending order of the order ids.
Order the query result tuples by subject ascending.

```
select subject, sum(quantity) as totalSold,  
       listagg(orderId, ', ' ) within group (order by orderId) as orders  
from Book natural join OrderDetails  
group by subject  
order by subject;
```

Query result

subject	totalSold	orders
Art	1115	1, 2, 4, 8, 11, 12, 14, 16, 17, 19, 20, 22, 24, 25
Business	1480	1, 3, 4, 6, 8, 9, 12, 13, 15, 18, 19, 22, 23, 24
Fiction	764	3, 4, 5, 7, 9, 10, 20, 21, 22, 23, 25

Is it necessary to join with BookOrder?

No! Why?

Book and OrderDetails can be joined directly on bookId.

EXERCISE 7

Find each book's rank with respect to the total number of books sold.
Include in the query result tuples attributes title,
author name (first followed by last) and book rank.
Order the query result tuples first by rank ascending
and then by title ascending.

```
select title, firstName || ' ' || lastName as author,  
       dense_rank() over (order by sum(quantity) desc nulls last) as bookRank  
from Author natural join Book natural left outer join OrderDetails  
group by title, firstName, lastName, bookId  
order by bookRank, title;
```

EXERCISE 7 (CONTD)

Query result

title	author	bookRank
Business Success	Donald Green	1
Purple Cow	Seth Godin	2
The Art of Negotiating	Pied Piper	3
Prey	Michael Crichton	4
Inferno	Dan Brown	5
Angels and Demons	Dan Brown	6
Origin of Art	Pied Piper	7
Woman in Art	Hillary Blue	7
Appreciating Art	Hillary Blue	8
Soul of a Nation	Mark Godin	8
The Lean Startup	Eric Ries	8
Negotiating Art	Pied Piper	9
Exodus Blue	Michael Crichton	10
Machinehood	Michael Crichton	11
Original Thinking	Dan Brown	12
Congo	Michael Crichton	13
The Women	Kristin Hannah	14

EXERCISE 8

Find the average total number of business books sold for each year as well as for all previous years truncated to one decimal place. Order the query result tuples by order year ascending.

```
select orderYear,  
       trunc(avg(sum(quantity)) over (order by orderYear  
                                     rows unbounded preceding),1) as avgSold  
from Book natural join BookOrder natural join OrderDetails  
where subject='Business'  
group by orderYear;
```

Query result

orderYear	avgSold
2022	80
2023	313
2024	342
2025	370